

Appendix

A Details of Markov Decision Process-based Reasoning Agent

To systematically capture the sequential dependencies inherent in SQL rewriting, we cast the process as a Markov Decision Process (MDP) $(\mathcal{S}, \mathcal{A}, T, r, \gamma)$. In this formulation:

States. Each $S_t \in \mathcal{S}$ encodes the current SQL query after t rewrite steps, including any injected hints or syntax repairs. The initial state is the user’s original query:

$$S_0 = Q_{\text{orig}}.$$

Actions. The action space $\mathcal{A}$ comprises

- *Refinement actions* R_t : targeted rewrites such as join reordering, predicate pushdown, common-table-expression (CTE) extraction, or hint injection.
- A *terminal action* A : emit the final rewritten query and proceed to verification.

Transitions. Applying R_t to S_t yields

$$S_{t+1} = T(S_t, R_t) = S_t + R_t,$$

where the “+” operator denotes syntactic transformation of the SQL AST. Choosing A halts further rewrites:

$$S_{t+1} = T(S_t, A) = S_t + A.$$

The resulting deterministic transition is:

$$S_{t+1} = \begin{cases} T(S_t, R_t) = S_t + R_t, & \text{if a refinement is applied,} \\ T(S_t, A) = S_t + A, & \text{if rewriting terminates.} \end{cases}$$

Reward. To guide the search toward efficient rewrites, we define a domain-specific *reward function*:

$$r(S_t, R_t) = \text{Cost}(S_t) - \text{Cost}(S_{t+1}),$$

where $\text{Cost}(\cdot)$ is the reasoning LLM estimated execution cost (via EXPLAIN). Positive reward indicates an estimated performance gain.

Policy. The agent’s behavior is governed by a policy

$$\pi_\theta(a | S_t) = \text{LLM}_\theta(a | \text{prompt}(S_t)),$$

implemented by a reasoning-specialized LLM (Deepseek-R1). At each step, the LLM is prompted with the current query state, schema statistics, and recent rewrite history, and produces a probability distribution over $\mathcal{A}$.

Objective. Under discount factor $\gamma \in (0, 1]$, we seek the policy that maximizes expected cumulative reward:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{k=0}^{T-1} \gamma^k r(S_k, a_k) \right].$$

Equivalently, the optimal Q-function satisfies the Bellman equation:

$$Q^*(S_t, a_t) = r(S_t, a_t) + \gamma \max_{a'} Q^*(S_{t+1}, a').$$

Rationale. By framing query rewriting as an MDP, we obtain:

- (1) *Cumulative optimization*: the agent evaluates rewrites on their long-term impact, avoiding local gains that hinder subsequent optimizations.

- (2) *Explicit dependency tracking*: each S_t retains full query context, ensuring that later actions (e.g., predicate pushdowns) respect earlier structural changes.
- (3) *LLM-aligned planning*: Deepseek-R1’s chain-of-thought generation naturally corresponds to sampling from π_θ over multiple timesteps, allowing it to propose coherent rewrite trajectories rather than isolated edits.

By treating each rewrite as a state transition, we make explicit how one transformation *enables* or *rules out* subsequent optimizations. To guide the search toward efficient and semantically sound plans, we define a hybrid *reward function*

$$r(S_t, R_t) = \alpha (\text{Cost}(S_t) - \text{Cost}(S_{t+1})) + (1 - \alpha) \text{Score}(S_t, R_t),$$

where:

- $\text{Cost}(\cdot)$ is the optimizer’s estimated execution cost (via EXPLAIN).
- $\text{Score}(S_t, R_t) \in [0, 1]$ is a semantic-and-plan coherence score produced by the reasoning LLM.

Here $\alpha \in [0, 1]$ balances between purely cost-driven improvements and the LLM’s holistic judgment. Intuitively, this hybrid reward leverages the optimizer to quantify immediate cost gains while relying on the LLM to correct for estimation biases and ensure semantic fidelity.

B Rewrite Examples Beyond Rules

Existing rule sets are inherently limited and can be suboptimal. In contrast, our framework leverages the agents to explore and discover more effective rewrite strategies—delivering performance gains that traditional rule-based methods cannot match. Below we present representative examples:

P1: Normalizes arithmetic predicates and converts redundant GROUP BY to ‘SELECT DISTINCT’.

By inverting the join expressions ($\text{DEPTNO} - 10 = \text{SAL} + 1$) into a simpler filter ($\text{SAL} = \text{DEPTNO} - 11$), then replacing the grouped projection of $\text{SAL} + 1$ per DEPTNO with a `SELECT DISTINCT` DEPTNO on the simplified predicate, thereby eliminating both the derived-table grouping and the extra computed columns.

Original Query (34.91s):

```
SELECT t4.NAME
FROM (
  SELECT NAME,
         DEPTNO,
         DEPTNO - 10 AS DEPTNOMINUS
  FROM DEPT
) AS t4
INNER JOIN (
  SELECT DEPTNO,
         SAL + 1 AS f9
  FROM EMP
  GROUP BY DEPTNO, SAL + 1
) AS t6
ON t4.DEPTNOMINUS = t6.f9
```

AND t4.DEPTNO = t6.DEPTNO;

Rewritten Query (0.74s):

```
SELECT t4.NAME
FROM (
  SELECT NAME,
         DEPTNO,
         DEPTNO - 10 AS DEPTNOMINUS
  FROM DEPT
) AS t4
INNER JOIN (
  SELECT DEPTNO,
         SAL + 1 AS f9
  FROM EMP
  GROUP BY DEPTNO, SAL + 1
) AS t6
ON t4.DEPTNOMINUS = t6.f9
AND t4.DEPTNO = t6.DEPTNO;
```

P2: Applies subquery flattening and aggregation elimination

Removing the two grouped derived tables and replacing them with a single 'EXISTS' predicate plus 'DISTINCT', thereby collapsing the LEFT JOIN+GROUP BY into a simple filter on the base table.

Original Query (38.01s):

```
SELECT t6.ENAME
FROM (
  SELECT JOB
  FROM EMP
  WHERE ENAME = 'A'
  GROUP BY JOB
) AS t5
LEFT JOIN (
  SELECT ENAME, JOB
  FROM EMP
  GROUP BY ENAME, JOB
) AS t6
ON t5.JOB = t6.JOB
GROUP BY t6.ENAME;
```

Rewritten Query (1.86s):

```
SELECT DISTINCT E.ENAME
FROM EMP E
WHERE EXISTS (
  SELECT 1
  FROM EMP A
  WHERE A.ENAME = 'A'
        AND A.JOB = E.JOB
);
```

P3: Eliminates subqueries and joins via tautology removal

Recognizes that both derived tables filter on the same predicates ('DEPTNO = 20 AND SAL > 1000') and that the LEFT JOIN+GROUP BY merely re-projects SAL values, so it collapses the logic into a single scan with the original filters and removes all unnecessary

grouping and joins.

Original Query (1.99s):

```
SELECT t0.SAL
FROM (
  SELECT EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM,
         SLACKER, SAL AS SAL0, SAL = 4 AS f9
  FROM EMPNULLABLES
  WHERE DEPTNO = 20 AND SAL > 1000
) AS t0
INNER JOIN (
  SELECT t2.SAL, t5.SAL AS SAL0, t5.f9
  FROM (
    SELECT EMPNO, ENAME, JOB, MGR, HIREDATE,
           SAL, COMM, SLACKER
    FROM EMPNULLABLES
    WHERE DEPTNO = 20 AND SAL > 1000
  ) AS t2
  INNER JOIN (
    SELECT SAL, SAL = 4 AS f9
    FROM EMPNULLABLES
    WHERE DEPTNO = 20 AND SAL > 1000
    GROUP BY SAL, SAL = 4
  ) AS t5
  ON t2.SAL = t5.SAL OR t5.f9
  GROUP BY t2.SAL, t5.SAL, t5.f9
  HAVING t5.SAL = t2.SAL
) AS t8
ON t0.SAL = t8.SAL0 AND t0.f9 = t8.f9;
```

Rewritten Query (1.11s):

```
SELECT SAL
FROM EMPNULLABLES
WHERE DEPTNO = 20
      AND SAL > 1000;
```

P4: Applies nested-subquery flattening and redundant

Join elimination—recognizes that all derived tables apply the same predicates ('DEPTNO = 20 AND SAL > 1000') and simply project 'SAL', so it collapses the nested SELECTs and INNER JOIN+GROUP BY into a single filtered scan on 'EMPNULLABLES'.

Original Query (2.07s):

```
SELECT t12.SAL
FROM (
  SELECT *
  FROM (
    SELECT EMPNO, ENAME, JOB, MGR, HIREDATE,
           SAL, COMM, SLACKER,
           SAL AS SAL0, SAL = 4 AS f9
    FROM EMPNULLABLES
    WHERE DEPTNO = 20 AND SAL > 1000
  ) AS t11
  WHERE SAL > 1000
) AS t12
INNER JOIN (
```

```

SELECT t14.SAL, t17.SAL AS SAL0, t17.f9
FROM (
  SELECT EMPNO, ENAME, JOB, MGR, HIREDATE,
  SAL, COMM, SLACKER
  FROM EMPNULLABLES
  WHERE DEPTNO = 20 AND SAL > 1000
) AS t14
INNER JOIN (
  SELECT SAL, SAL = 4 AS f9
  FROM EMPNULLABLES
  WHERE DEPTNO = 20 AND SAL > 1000
  GROUP BY SAL, SAL = 4
) AS t17
  ON t14.SAL = t17.SAL OR t17.f9
GROUP BY t14.SAL, t17.SAL, t17.f9
HAVING t17.SAL = t14.SAL
) AS t20
  ON t12.SAL = t20.SAL0
AND t12.f9 = t20.f9;

```

Rewritten Query (1.10s):

```

SELECT SAL
FROM EMPNULLABLES
WHERE DEPTNO = 20
  AND SAL > 1000;

```

P5: Applies join-elimination and aggregation unification

Recognizing that both derived tables group by the same key 'SAL' and then inner-join on 'SAL' merely re-produces the same aggregation, so it collapses the two into one single GROUP BY, removing the redundant join.

Original Query (3.46s):

```

SELECT SUM(CAST(t3.f1 * t4.SAL AS INTEGER))
FROM (
  SELECT SAL, COUNT(*) AS f1
  FROM EMP
  GROUP BY SAL
) AS t3
INNER JOIN (
  SELECT SAL
  FROM EMP
  GROUP BY SAL
) AS t4
  ON t3.SAL = t4.SAL;

```

Rewritten Query (1.80s):

```

SELECT SUM(CAST(f1 * SAL AS INTEGER))
FROM (
  SELECT SAL, COUNT(*) AS f1
  FROM EMP
  GROUP BY SAL
) AS t;

```

P6: Applies subquery flattening and EXISTS-to-IN unnesting

Eliminating the nested derived tables and GROUP BY on a constant expression and converting the correlated EXISTS into a simple IN predicate at the top level.

Original Query (0.29s):

```

SELECT NAME
FROM (
  SELECT DEPTNO, NAME
  FROM DEPT
  WHERE EXISTS (
    SELECT 1
    FROM (
      SELECT 2 * DEPTNO AS f0
      FROM DEPT
      GROUP BY 2 * DEPTNO
    ) AS t5
    WHERE DEPTNO = t5.f0
  )
) AS t2;

```

Rewritten Query (0.13s):

```

SELECT NAME
FROM DEPT
WHERE DEPTNO IN (
  SELECT 2 * DEPTNO
  FROM DEPT
);

```

P7: Applies limit-pushdown through subquery reordering

By moving the FETCH FIRST 10 ROWS ONLY inside the EMP scan, the optimizer retrieves only the top-10 highest-salary employees before performing the join, rather than joining the entire EMP and DEPT tables and then limiting.

Original Query (5.56s):

```

SELECT *
FROM EMP
LEFT JOIN (SELECT * FROM DEPT) AS t
  ON EMP.DEPTNO = t.DEPTNO
ORDER BY EMP.SAL
FETCH NEXT 10 ROWS ONLY;

```

Rewritten Query (1.12s):

```

SELECT e.*, t.*
FROM (
  SELECT *
  FROM EMP
  ORDER BY SAL
  FETCH FIRST 10 ROWS ONLY
) e
LEFT JOIN DEPT t
  ON e.DEPTNO = t.DEPTNO
ORDER BY e.SAL;

```

P8: Normalizes quantified predicates by converting '> SOME (subquery)' into a simple comparison against 'MIN(...)'

Rewriting $x > \text{SOME}(\text{subquery})$ as $x > (\text{SELECT MIN}(\dots) \text{ FROM subquery})$, which eliminates the quantifier and lets the optimizer treat it as a simple filter.

Original Query (103.86s):

```
SELECT *
FROM emp e1
WHERE e1.empno > SOME (
    SELECT deptno
    FROM dept
);
```

Rewritten Query (19.44s):

```
SELECT *
FROM emp e1
WHERE e1.empno > (
    SELECT MIN(deptno)
    FROM dept
);
```

P9: Join-elimination and DISTINCT projection

Recognizes that a full outer join grouping on identical columns adds no new information beyond the distinct values themselves, so it drops the join and grouping entirely, replacing it with a single SELECT DISTINCT mgr over the base table.

Original Query (>300s):

```
SELECT
    e.mgr,
    d.mgr
FROM
    emp AS e
    FULL OUTER JOIN emp AS d
        ON e.mgr = d.mgr
GROUP BY
    e.mgr,
    d.mgr;
```

Rewritten Query (10.74s):

```
SELECT DISTINCT mgr, mgr
FROM emp;
```

P10: Applies multi-CTE filter materialization and decorrelation of EXISTS subqueries

First isolating qualifying orders and lineitems into CTEs ('filtered_orders', 'lineitem_filtered') so that the status and date filters are applied once, and then referencing these CTEs in the main query and in the 'EXISTS'/'NOT EXISTS' checks. This lifts correlated predicates out of the main scan, avoids repeated evaluation, and lets the optimizer plan each CTE independently.

Original Query (>300s):

```
SELECT
```

```
    s.s_name,
    COUNT(*) AS numwait
FROM
    supplier AS s
    JOIN nation AS n
        ON s.s_nationkey = n.n_nationkey
    JOIN lineitem AS l1
        ON s.s_suppkey = l1.l_suppkey
    JOIN orders AS o
        ON l1.l_orderkey = o.o_orderkey
WHERE
    o.o_orderstatus = 'F'
    AND l1.l_receiptdate > l1.l_commitdate
    AND EXISTS (
        SELECT 1
        FROM lineitem AS l2
        WHERE l2.l_orderkey = l1.l_orderkey
            AND l2.l_suppkey <> l1.l_suppkey
    )
    AND NOT EXISTS (
        SELECT 1
        FROM lineitem AS l3
        WHERE l3.l_orderkey = l1.l_orderkey
            AND l3.l_suppkey <> l1.l_suppkey
            AND l3.l_receiptdate > l3.l_commitdate
    )
    AND n.n_name = 'SAUDI ARABIA'
GROUP BY
    s.s_name
ORDER BY
    numwait DESC,
    s.s_name
LIMIT 100;
```

Rewritten Query (86.50s):

```
WITH filtered_orders AS (
    SELECT o_orderkey
    FROM orders
    WHERE o_orderstatus = 'F'
),
lineitem_filtered AS (
    SELECT
        l_orderkey,
        l_suppkey,
        l_receiptdate,
        l_commitdate
    FROM lineitem
    WHERE l_orderkey IN (
        SELECT o_orderkey FROM filtered_orders
    )
),
filtered_suppliers AS (
    SELECT
        s_suppkey,
        s_name
    FROM supplier
```

```

        JOIN nation ON s_nationkey = n_nationkey
        WHERE n_name = 'SAUDI ARABIA'
    )
    SELECT
        s.s_name,
        COUNT(*) AS numwait
    FROM
        filtered_suppliers s
        JOIN lineitem_filtered l1
            ON s.s_suppkey = l1.l_suppkey
    WHERE
        l1.l_receiptdate > l1.l_commitdate
        AND EXISTS (
            SELECT 1
            FROM lineitem_filtered l2
            WHERE
                l2.l_orderkey = l1.l_orderkey

```

```

                AND l2.l_suppkey <> l1.l_suppkey
        )
        AND NOT EXISTS (
            SELECT 1
            FROM lineitem_filtered l3
            WHERE
                l3.l_orderkey = l1.l_orderkey
                AND l3.l_suppkey <> l1.l_suppkey
                AND l3.l_receiptdate > l3.l_commitdate
        )
    GROUP BY
        s.s_name
    ORDER BY
        numwait DESC,
        s.s_name
    LIMIT 100;

```